

Курс «Тестирование ПО»

Семинар
Анализ требований и тест-дизайн

Что такое требование?

Требование — это документально зафиксированное условие или характеристика, которой должна обладать система, чтобы удовлетворить потребность стейкхолдера

Кривая стоимости дефектов

Стадия обнаружения	Стоимость исправления
Этап требований (ревью)	×1
Этап проектирования	×5
Этап кодирования	×10
Этап тестирования	×50
Продакшен	×200+

Зачем нужен тест-дизайн?

Тест-дизайн даёт три результата:

1. Меньше тестов при том же покрытии.
2. Предсказуемая полнота.
3. Обоснование трудозатрат.

Определение по ISTQB

***Тест-дизайн** — это процесс создания и определения тестовых сценариев и конкретных тест-кейсов на основе анализа тест-базиса (требований, спецификаций, моделей).*

План семинара

Блок	Что делаем
Блок 1	Введение
Блок 2	Анализ требований: виды, критерии качества, ревью спецификации
Блок 3	Пять техник тест-дизайна на сквозном примере
Блок 4	Метрики покрытия и приоритизация
Блок 5	Итоги и вопросы

Иерархия требований

Тип	Что описывает	Пример из сквозного задания
Бизнес-требование	Зачем нужна фича, бизнес-цель	«Увеличить конверсию новичков приветственной скидкой»
Пользовательское	Что делает пользователь	«Покупатель вводит промокод и получает скидку»
Функциональное	Как система реагирует	«При вводе VIP25 и сумме $\geq 5\,000$ система применяет 25% скидку»
Нефункциональное	Качественные характеристики	«Расчёт стоимости выполняется < 1 с; промокод не логируется в открытый лог»
Техническое / ограничение	Среда, интеграции	«Промокоды хранятся в сервисе Billing, синхронизация раз в 5 мин»

Мнемоника INVEST + ATM

- I — Independent (независимое).
- N — Negotiable (обсуждаемое).
- V — Valuable (ценное).
- E — Estimable (оценимое).
- S — Small (компактное).
- T — Testable (тестируемое).
- A — Ambiguous-free (однозначное).
- M — Modifiable (изменяемое).

Антипаттерны требований

- Антипаттерн 1: «Скидка должна применяться корректно» (п. 2 спецификации).
- Антипаттерн 2: «Система должна работать быстро и быть удобной» (п. 7 спецификации).
- Антипаттерн 3: «VIP25 — 25% для клиентов со статусом VIP и новичков» (п. 2 спецификации).
- Антипаттерн 4: «Максимальная сумма — 100 000 ₽» (п. 1 спецификации).

Чек-лист ревью

- Есть ли явно заданные границы и диапазоны?
- Указано ли поведение для пустых / нулевых / отрицательных значений?
- Что происходит при ошибке?
- Есть ли зависимость от других требований или состояний?
- Описаны ли права доступа?
- Какое поведение по умолчанию?
- Что считается «успехом» (acceptance criteria)?
- Совместимо ли требование с уже существующей функциональностью?

Ревью спецификации

№	Проблема в формулировке	Уточняющий вопрос аналитику
1	«Сумма максимум 100 000 ₽»	Граница включительно? Это до или после применения скидки?
2	«Новый клиент»	Что считать новым? Никогда не заказывал? Заказывал, но отменил?
3	«Просроченный промокод»	Где хранится срок? По какому часовому поясу?
4	«Доставка Почтой — до 5 000 ₽»	Включительно 5 000? Это после скидки или до?
5	«Недостаточно средств»	Баланс резервируется в момент создания заказа или оплаты? Что при мультивалюте?
6	Не описано поведение при вводе промокода в нижнем регистре	Чувствителен ли к регистру?
7	Не описаны ошибки ввода нечисловой суммы	Какая валидация на клиенте и сервере?

Теория: классы эквивалентности

Правила разбиения:

1. Валидные классы — система принимает значение, обрабатывает корректно.
2. Невалидные классы — система отклоняет значение, выдаёт ошибку.
3. Из каждого класса берём **одно** тестовое значение — любое.

Пример:

поле «Сумма заказа»

Класс	Тип	Тестовое значение	Ожидаемый результат
$sum \leq 0$	невалидный	0, -500	Ошибка «Сумма должна быть положительной»
$0 < sum \leq 100\,000$	валидный	4 999	Принято
$sum > 100\,000$	невалидный	150 000	Ошибка «Превышен максимум»
не число	невалидный	abc, 12,5 (запятая)	Ошибка «Некорректный формат»

Пример: поле «Промокод»

Класс	Тип	Значение
WELCOME10, новый клиент	валидный	скидка 10%
VIP25, VIP, sum $\geq$ 5 000	валидный	скидка 25%
VIP25, VIP, sum < 5 000	невалидный	ошибка условий
VIP25, не VIP	невалидный	ошибка
Несуществующий код	невалидный	ошибка
Пустое поле	валидный	скидки нет

Теория: граничные значения

Классическое правило: для каждой границы проверяем шесть значений: $\text{min}-1$, min , $\text{min}+1$, $\text{max}-1$, max , $\text{max}+1$.

Для строк и коллекций границы — это длина: пустая строка, строка из одного символа, максимальная длина, на один больше максимума.

Пример: границы в задании

Поле	Граница	Тесты
Сумма (1 ... 100 000)	нижняя	0, 1, 2
Сумма	верхняя	99 999, 100 000, 100 001
VIP-скидка ($\text{sum} \geq 5\,000$)	порог	4 999, 5 000, 5 001
Курьер ($\text{sum} \geq 1\,000$)	порог	999, 1 000, 1 001
Почта ($\text{sum} \leq 5\,000$)	порог	4 999, 5 000, 5 001 (последнее — невалидно)
Промокод (строка)	длина	пусто, WELCOME10 (9 символов), строка 1000 символов

Теория: таблицы решений

Структура таблицы:

- Строки — условия (C1, C2, C3 ...).
- Колонки — комбинации значений условий.
- Нижние строки — действия (что система делает).
- Каждая колонка = один тест-кейс.

Пример: Расчет скидки

№	C1 новый	C2 VIP	C3 ≥ 5000	C4 промокод	Действие
1	Y	N	*	WELCOME10	10%
2	N	Y	Y	VIP25	25%
3	N	Y	N	VIP25	Ошибка «Мин. сумма 5 000»
4	N	N	*	VIP25	Ошибка «Только для VIP»
5	*	*	*	несуществующий	Ошибка «Промокод недействителен»
6	*	*	*	пусто	Без скидки
7	Y	N	*	VIP25	Ошибка «Только для VIP»
8	N	Y	Y	WELCOME10	Ошибка «Только для новых»

* означает, что значение условия не влияет на результат в данной комбинации

Теория: переходы состояний

Элементы модели:

- Состояния
- Переходы
- События
- Условия
- Действия

Виды покрытия:

- 0-switch (покрытие переходов)
- 1-switch (покрытие пар переходов)
- Все состояния
- Негативные переходы

Пример: Расчет скидки

Жизненный цикл заказа

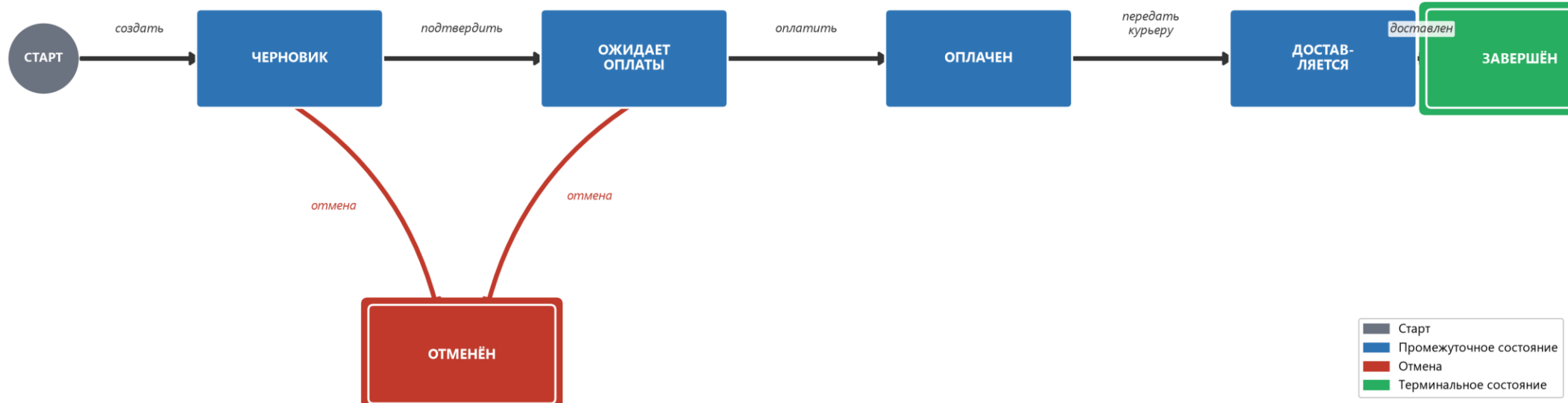


Таблица переходов (0-switch coverage)

Из состояния	Событие	→ Состояние
СТАРТ	создать	ЧЕРНОВИК
ЧЕРНОВИК	подтвердить	ОЖИДАЕТ ОПЛАТЫ
ЧЕРНОВИК	отмена	ОТМЕНЁН
ОЖИДАЕТ ОПЛАТЫ	оплатить	ОПЛАЧЕН
ОЖИДАЕТ ОПЛАТЫ	отмена	ОТМЕНЁН
ОПЛАЧЕН	передать курьеру	ДОСТАВЛЯЕТСЯ
ДОСТАВЛЯЕТСЯ	доставлен	ЗАВЕРШЁН

Негативные переходы

- **ОПЛАЧЕН → отмена.** Можно ли отменить оплаченный заказ? Если да — как возвращать деньги? Спецификация не описывает возврат, значит, кнопка «Отмена» должна быть недоступна или выдавать ошибку.
- **ЗАВЕРШЁН → любой.** Завершённый заказ — терминальное состояние. Никаких переходов быть не должно. Проверяем, что API не позволяет изменить статус.
- **ОТМЕНЁН → ОЖИДАЕТ ОПЛАТЫ.** Можно ли «оживить» отменённый заказ? Спецификация не описывает восстановление — значит, переход запрещён.
- **ЧЕРНОВИК → ДОСТАВЛЯЕТСЯ.** Перескок через статусы. Если система позволяет — это серьёзный дефект бизнес-логики.

1-switch coverage

1-switch coverage — покрытие пар последовательных переходов. Для критичных участков (например, счастливый путь заказа) проверяем, что система корректно проходит всю цепочку без сбоев между переходами:

Цепочка: ЧЕРНОВИК → ОЖИДАЕТ ОПЛАТЫ → ОПЛАЧЕН → ДОСТАВЛЯЕТСЯ → ЗАВЕРШЁН.

Каждая пара — отдельный тест: создаём заказ, подтверждаем, оплачиваем, проверяем, что данные сохранились между переходами (сумма, адрес, промокод не «потерялись»).

Плюс пары с отменой: ЧЕРНОВИК → ОТМЕНЁН (проверяем, что после отмены нельзя подтвердить). ОЖИДАЕТ ОПЛАТЫ → ОТМЕНЁН (проверяем, что отмена из ожидания действительно отменяет, а не «зависает»).

Теория: Попарное тестирование

Pairwise (all-pairs) гарантирует: каждая пара значений любых двух параметров встретится хотя бы в одном тесте. Это сокращает количество тестов в 5-10 раз при сохранении покрытия пар.

Пример: параметры формы

Параметр	Значения
Тип клиента	Новый, Обычный, VIP
Сумма	< 1 000, 1 000–5 000, > 5 000
Промокод	нет, WELCOME10, VIP25, неверный
Доставка	Самовывоз, Курьер, Почта

Пример вывода

Тест	Тип клиента	Сумма	Промокод	Доставка
1	Новый	< 1 000	WELCOME10	Самовывоз
2	Новый	1 000–5 000	VIP25	Курьер
3	VIP	> 5 000	VIP25	Почта
4	Обычный	1 000–5 000	неверный	Курьер
5	VIP	< 1 000	нет	Самовывоз
...	...	...	...	...

Матрица трассировки

Требование	Тест-кейсы	Статус покрытия
FR-1: сумма 1...100 000	ТС-001...ТС-006 (границы)	Покрыто
FR-2: скидка WELCOME10	ТС-010, ТС-015	Покрыто
FR-3: VIP25 при $\geq 5\,000$	ТС-011, ТС-012	Покрыто
NFR-1: время расчёта $< 1\text{ с}$	—	Нет теста

Виды покрытия

- **Покрытие требований (Requirements Coverage)** = % требований, для которых есть хотя бы один тест-кейс. Базовый показатель для QA. Если 28 из 30 требований покрыты — покрытие 93%.
- **Покрытие кода (Code Coverage)** — метрика разработчика: какие строки, ветви, пути кода выполнены во время тестов. Не заменяет покрытие требований: 100% покрытие кода не означает, что все требования проверены.
- **Покрытие переходов** — для state-based тестирования: % переходов в диаграмме состояний, покрытых тестами.

Приоритизация

Risk-Based Testing. Фокус на функциях с высоким риском. Риск = вероятность дефекта × ущерб. В нашем примере:

- Расчёт скидки и проверка баланса — **высокий риск** (деньги, репутация). Тестируем первыми.
- Валидация формата поля «Сумма» — **низкий риск** (не влияет на деньги). Можно отложить.

MoSCoW. Делим тесты на:

- **Must** — обязательно, без них релиз невозможен.
- **Should** — важно, но можно пропустить при нехватке времени.
- **Could** — желательно, если осталось время.
- **Won't** — в этот релиз не тестируем.

Последовательность запуска при регресе:

- Happy path — основные сценарии, которые используют 80% пользователей.
- Альтернативные пути — варианты happy path.
- Негативные тесты — ошибки и граничные случаи.
- Краевые и редко используемые — если осталось время.